Get unlimited access to the best of Medium for less than $1/week. Become a member

PETL Powerhouse: Exploring the Versatility of Python's ETL tool 🏭

Manoj Das · Follow
5 min read · Jun 28, 2023

22

> *What is PETL? ETL functions in Python and Utility function in Python provided by PETL. An example of how to use PETL.*

Photo by Karl Hörnfeldt on Unsplash

PETL refers to the Python Extract, Transform, Load (ETL) library. It is a lightweight and powerful tool designed for working with tabular data, such as CSV files, Excel spreadsheets, and SQL databases. PETL provides a simple and intuitive interface for performing common ETL tasks, such as filtering, sorting, joining, aggregating, and transforming data.

History

The history of PETL can be traced back to its initial release in 2011 by Christian Alis, a data enthusiast and software developer. petl was developed to address the need for a simple and flexible ETL library in Python.

It was inspired by similar ETL tools in other programming languages and aimed to provide a similar capability in the Python ecosystem. The library was designed with a focus on ease of use, performance, and a functional programming style.

Over the years, PETL gained popularity among Python developers working with tabular data due to its simplicity and powerful features. It became widely adopted in various industries and data-centric domains, including data engineering, data analysis, and scientific research.

The PETL library evolved with regular updates and enhancements, expanding its capabilities and improving its performance. It received contributions from the open-source community, which helped to enrich its functionality and address user needs.

source

Advantages

PETL offers several advantages that make it a powerful and popular choice for working with tabular data in Python. Here are some key advantages of using PETL:

Simplicity: PETL provides a simple and intuitive API that makes it easy to perform common ETL tasks. The library follows a functional programming style, allowing you to chain operations together to create data processing pipelines. This approach promotes code readability and maintainability.

Flexibility: PETL is designed to work with various data sources and formats, including CSV files, Excel spreadsheets, SQL databases, and more. It provides a consistent interface for working with different data structures, enabling seamless integration with diverse data pipelines.

Performance: PETL is optimized for performance and efficiency. It leverages Python's underlying data processing capabilities, such as generators and iterators, to handle large datasets efficiently. The library employs lazy evaluation, which means that data is processed on-demand, reducing memory usage and improving overall performance.

Extensibility: PETL is highly extensible, allowing you to write custom functions and operations to suit your specific data processing needs. You can easily combine PETL with other Python libraries and tools to create comprehensive data pipelines.

Data Integrity: PETL includes robust data validation and error handling mechanisms. It provides features like data type validation, field length checks, and error logging, helping you ensure data integrity during the ETL process.

Integration with Python Ecosystem: PETL seamlessly integrates with other popular libraries and tools in the Python ecosystem. You can combine it with libraries like pandas, NumPy, and scikit-learn for advanced data analysis and machine learning tasks. Additionally, PETL works well with database libraries such as SQLAlchemy, enabling easy integration with SQL databases.

Active Community and Documentation: PETL has an active and supportive community of users and contributors. The library is well-documented, providing comprehensive guides, examples, and API references. The community actively maintains and updates PETL , ensuring the library remains relevant and up-to-date.

ETL Functions

Extract Functions:

fromcsv(): Extracts data from a CSV file and returns a petl table.

fromjson(): Extracts data from a JSON file and returns a petl table.

fromdb(): Extracts data from a SQL database and returns a petl table.

fromxml(): Extracts data from an XML file and returns a petl table.

fromtext(): Extracts data from a text file and returns a petl table.

Transform Functions:

cut(): Selects specific columns from a table and returns a new table.

select(): Filters rows from a table based on a condition and returns a new table.

aggregate(): Performs aggregation operations, such as sum, count, or average, on one or more columns of a table and returns a new table.

join(): Combines multiple tables based on common keys and returns a new table with joined data.

addfield(): Adds a new column to a table based on a transformation or computation.

rename(): Renames columns in a table.

Load Functions:

tocsv(): Writes a petl table to a CSV file.

tojson(): Writes a petl table to a JSON file.

todb(): Writes a petl table to a SQL database.

toxml(): Writes a petl table to an XML file.

totext(): Writes a petl table to a text file.

Utility Functions

PETL provides several utility functions that can assist in various data processing tasks. Here are some notable utility functions available in petl:

head(table, n=10): Returns the first n rows from the input table. Useful for previewing data.

tail(table, n=10): Returns the last n rows from the input table. Helpful for inspecting the end of a table.

peek(table, n=10): Displays a preview of both the first and last n rows of the table. Useful for getting an overview of the data.

sample(table, n=10): Returns a random sample of n rows from the input table. Handy for generating a representative subset of data.

count(table): Returns the total number of rows in the input table.

distinct(table, key=None): Returns a table containing distinct rows based on the specified key column(s). If no key is provided, it returns unique rows across all columns.

split(table, key, sizes): Splits the input table into multiple tables based on the values in the specified key column. The sizes parameter defines the relative sizes of the resulting tables.

cutout(table, key): Removes the specified key column(s) from the input table and returns the modified table.

addrownumbers(table, start=1): Adds a new column to the table containing row numbers, starting from the specified start value.

progress(table, every=1000, length=None): Wraps an input table and displays progress updates every every rows processed. The length parameter allows setting a total row count for more accurate progress estimation.

Photo by Jason Blackeye on Unsplash

Installation

We can install PETL by running the following command:

```
pip install petl
```

A Simple Example

Let's say we have a CSV file containing sales data with the following columns: "OrderID", "Product", "Quantity", "Price". We want to perform the following ETL operations:

1. Load the data from the CSV file.
2. Filter out rows where the quantity is less than or equal to zero.
3. Calculate the total sales amount for each product.
4. Sort the data based on the total sales amount in descending order.
5. Save the transformed data to a new CSV file.

```
import petl as etl

# Load the data from CSV file
table = etl.fromcsv('sales_data.csv')

# Filter out rows with quantity <= 0
filtered_table = etl.select(table, lambda row: row['Quantity'] > 0)

# Calculate the total sales amount for each product
sales_table = etl.aggregate(filtered_table, key='Product', aggregation={'TotalSa

# Sort the data based on total sales amount
sorted_table = etl.sort(sales_table, key='TotalSales', reverse=True)

# Save the transformed data to a new CSV file
etl.tocsv(sorted_table, 'transformed_sales_data.csv')
```

— — —

Why did the ETL process go to therapy?

Because it was having trouble with its "PETL issues"!

🙂🙂🙂

Python Petl Etl Data Science Machine Learning

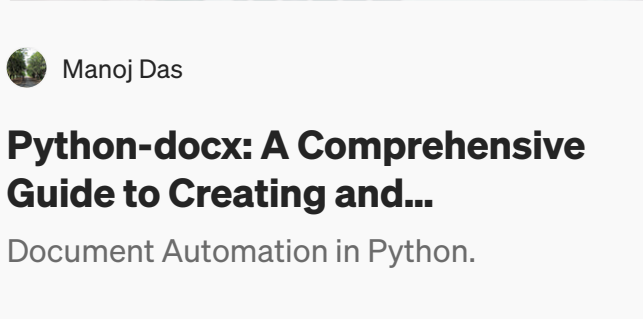
22

Written by Manoj Das
433 Followers · 2 Following
Follow

No responses yet

What are your thoughts? Respond

More from Manoj Das

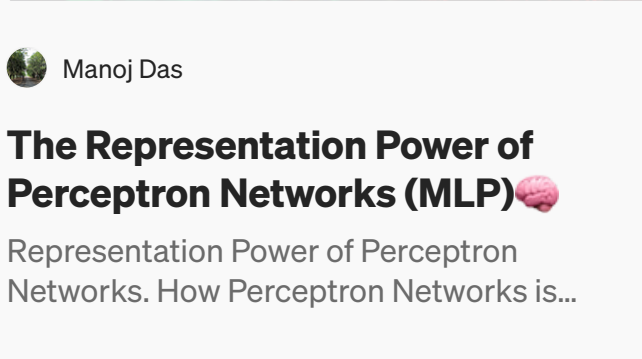


Manoj Das

Python-docx: A Comprehensive Guide to Creating and...

Document Automation in Python.

Sep 3, 2023 206 3



Manoj Das

The Representation Power of Perceptron Networks (MLP)🧠

Representation Power of Perceptron Networks. How Perceptron Networks is...

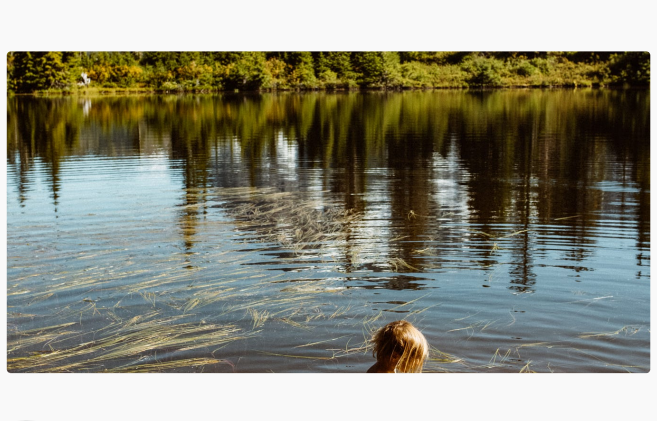
Jul 6, 2023 9

Manoj Das

Rembg: Effortlessly Remove Backgrounds in Python 📷

What is Rembg? How to use Rembg to remove background from image in Python?

Jun 16, 2023 150 2



Manoj Das

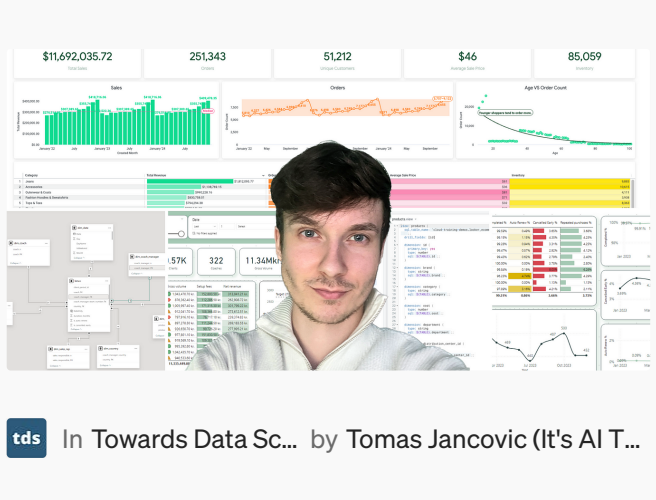
Taipy: Magical Python Library to Create Simple, Quick & Efficient...

What is Taipy in Python? A comprehensive guide to Taipy python library. Taipy vs...

Nov 25, 2023 402 3

See all from Manoj Das

Recommended from Medium

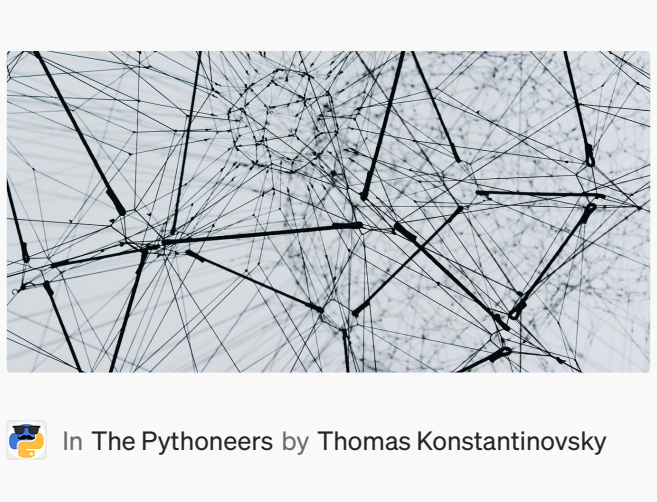


In Towards Data Sc... by Tomas Jancovic (It's AI T...

My Experience Switching From Power BI to Looker (as a Senior...

What you need to know before you switch from Power BI to Looker.

5d ago 225 4



In The Pythoneers by Thomas Konstantinovsky

Mastering the SEIR Model: A Comprehensive Guide with Pytho...

How to Simulate and Visualize Disease Spread Using Python: An Essential Model fo...

Sep 19, 2024 24

Lists

Predictive Modeling w/ Python
20 stories · 1791 saves

Coding & Development
11 stories · 982 saves

Practical Guides to Machine Learning
10 stories · 2169 saves

Natural Language Processing
1893 stories · 1553 saves

In Python in Plain English by Kiran Maan

Just STOP Checking For NONE Like This in Python!!

Instead, use this way...which I am gonna tell you.

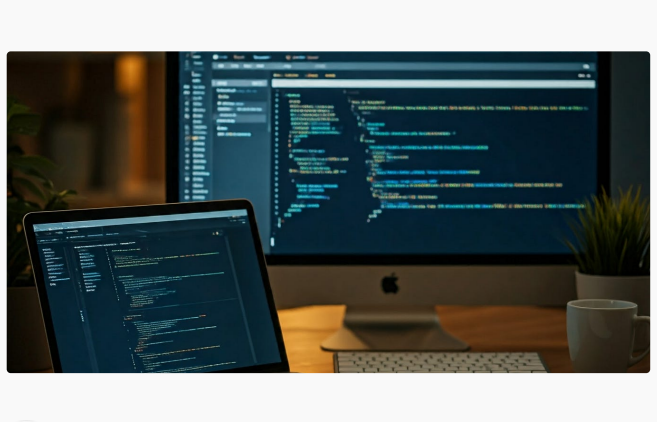
6d ago 262 7

Ibtissam Makdoun

TOP 5 Time Series Forecasting Libraries in 2025 for Python

(with Pros and GitHub Stars)

Jan 14 38

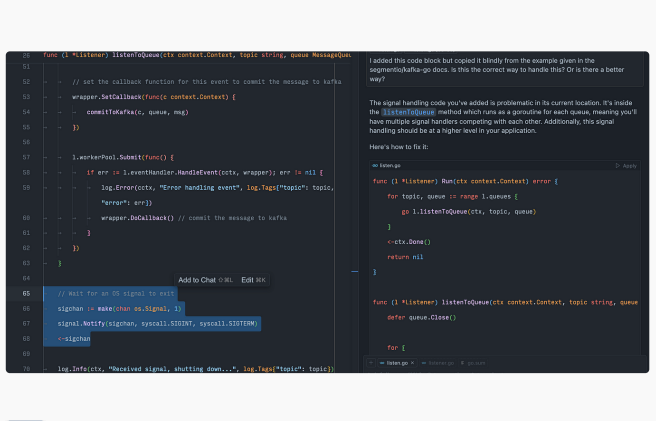


Bryan Antoine

Simplifying FastAPI Development with SQLModel: A Step-by-Step...

When developing APIs, a significant challenge developers face is dealing with...

Nov 28, 2024 6 1



In Level Up Coding by Jacob Bennett

The 5 paid subscriptions I actually use in 2025 as a Staff Software...

Tools I use that are cheaper than Netflix

Jan 7 6K 127

See more recommendations